PyTorch SimCLR Implementation Example

https://gemini.google.com/app/2448758aebf27a80

User prompt: simclr example

Response: Here is a complete, minimal implementation of **SimCLR** (Simple Framework for Contrastive Learning of Visual Representations) in **PyTorch**.

SimCLR relies on four key components:

1. **Stochastic Data Augmentation Module**: Generates two correlated views ($\tilde{x}_i$ and $\tilde{x}_j$) of the same image.
2. **Base Encoder ($f(\cdot)$)**: Extracts representation vectors from augmented images (e.g., a ResNet).
3. **Projection Head ($g(\cdot)$)**: An MLP that maps representations to the space where contrastive loss is applied.
4. **NT-Xent Loss (Normalized Temperature-scaled Cross Entropy)**: The contrastive loss that pulls positive pairs together and pushes negative pairs apart.

1. Data Augmentation Pipeline

SimCLR's performance depends heavily on the composition of data augmentations (specifically random crop, color distortion, and Gaussian blur).

```
import torch
import torchvision.transforms as T

class SimCLRAugmentations:
    def __init__(self, size=224):
        # SimCLR paper heavily emphasizes strong color jitter and blur
        color_jitter = T.ColorJitter(0.8, 0.8, 0.8, 0.2)

        self.transform = T.Compose([
            T.RandomResizedCrop(size=size, scale=(0.08, 1.0)),
            T.RandomHorizontalFlip(p=0.5),
            T.RandomApply([color_jitter], p=0.8),
            T.RandomGrayscale(p=0.2),
            T.GaussianBlur(kernel_size=int(size * 0.1) | 1, sigma=(0.1, 2.0)),
            T.ToTensor(),
            T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        ])

    def __call__(self, x):
        # Return two different augmented views of the same image
        return self.transform(x), self.transform(x)
```

2. SimCLR Model Architecture

Combines a standard backbone encoder with a non-linear projection head.

```
import torch.nn as nn
import torchvision.models as models

class SimCLR(nn.Module):
    def __init__(self, base_encoder='resnet18', projection_dim=128):
        super().__init__()

        # 1. Base Encoder (e.g., ResNet)
        encoder = getattr(models, base_encoder)(weights=None)
        self.encoder = nn.Sequential(*list(encoder.children())[:-1]) # Remove final FC layer
        encoder_dim = encoder.fc.in_features

        # 2. Non-linear Projection Head (MLP: encoder_dim -> 2048 -> projection_dim)
        self.projection_head = nn.Sequential(
            nn.Linear(encoder_dim, 2048),
            nn.ReLU(),
            nn.Linear(2048, projection_dim)
        )

    def forward(self, x):
        h = self.encoder(x).squeeze(-1).squeeze(-1) # Flatten spatial dimensions
```

```
        z = self.projection_head(h)
        return h, z
```

3. NT-Xent Loss Function

Calculates the normalized temperature-scaled cross-entropy loss across batch pairs.

```
import torch.nn.functional as F

class NTXentLoss(nn.Module):
    def __init__(self, temperature=0.5):
        super().__init__()
        self.temperature = temperature
        self.criterion = nn.CrossEntropyLoss(reduction="sum")

    def forward(self, z_i, z_j):
        batch_size = z_i.shape[0]

        # Normalize representations to unit hypersphere
        z_i = F.normalize(z_i, dim=1)
        z_j = F.normalize(z_j, dim=1)

        # Concatenate views: shape (2 * N, D)
        representations = torch.cat([z_i, z_j], dim=0)

        # Compute similarity matrix
        similarity_matrix = torch.matmul(representations, representations.T) / self.temperature

        # Mask out self-similarity scores
        mask = torch.eye(2 * batch_size, dtype=torch.bool, device=z_i.device)
        similarity_matrix.masked_fill_(mask, -float('inf'))

        # Positive pairs: i <-> i+N and i+N <-> i
        labels = torch.cat([
            torch.arange(batch_size, 2 * batch_size, device=z_i.device),
            torch.arange(batch_size, device=z_i.device)
        ], dim=0)

        loss = self.criterion(similarity_matrix, labels)
        return loss / (2 * batch_size)
```

4. Training Step Example

```
# Setup
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = SimCLR(base_encoder='resnet18').to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=3e-4, weight_decay=1e-4)
criterion = NTXentLoss(temperature=0.5)

# Dummy training loop iteration
model.train()
# images shape: [Batch_Size, 3, 224, 224]
# augment_fn returns two views: view_1, view_2
augment_fn = SimCLRAugmentations(size=224)

# Example batch processing
images = torch.randn(32, 3, 224, 224, device=device)
view_1, view_2 = augment_fn(images) # Typically done inside the Dataset __getitem__

optimizer.zero_grad()

_, z_i = model(view_1)
_, z_j = model(view_2)

loss = criterion(z_i, z_j)
loss.backward()
optimizer.step()

print(f"SimCLR Step Loss: {loss.item():.4f}")
```
